Why Atrium: one memory for you, your team and all your AI assistants

You already use AI assistants. They are good, and they are forgetful. Every chat starts from zero, every assistant is an island, and what one colleague worked out on Tuesday is lost to everybody else by Wednesday.

Atrium Tablinum (Atrium for short) is the shared memory your assistants were missing. It is a place where knowledge and approved working instructions are kept once and then used by everyone: by you, by your colleagues, and by whichever assistant each of you prefers, Claude, ChatGPT or any other that speaks MCP.

This document is for everyone who wonders whether it is worth the effort to start using it. It says what you get, what your team gets, what the whole organisation gets, and, honestly, what Atrium does not do.

Contents

- The problem
- The idea in one minute
- What it brings you
- What it brings your team
- One memory, many assistants
- What it brings the organisation
- A day with Atrium
- Why you can trust it
- How Atrium compares
- Try it today: your first ten minutes
- The honest part: what Atrium does not do
- What it takes to run
- Rolling it out
- Questions people ask

The problem

Think of last week and count the times when:

- you explained the same background to an assistant that you had explained before, in another chat;
- you asked a colleague something that someone had already answered, somewhere, once;
- a good prompt, checklist or procedure lived in one person's notes and nowhere else;
- you switched from one assistant to another and had to start over;
- a decision was made, and three months later nobody could say why;
- a colleague left, and part of what they knew left with them.

None of this is dramatic. All of it is a steady tax on everybody's day, and it grows with every person and every assistant you add. Assistants make individuals faster. Without a shared memory they do little to make the *team* faster, because each one only knows what its own user told it in the current chat.

The idea in one minute

Atrium is a store that your assistants can read from and write to, **when you ask them to**. It holds two kinds of content:

- **Knowledge**: facts, processes, decisions and know-how written as short Markdown entries. Your assistant searches them to answer questions with *your* organisation's facts instead of generic ones.
- **Skills**: approved working instructions ("how we review code", "how we answer a support ticket", "our weekly report"). Anyone can run one with a command such as `/weekly-report`, and every colleague gets the same, reviewed way of working.

Content lives in **spaces** with roles: open ones for everybody, closed ones for a team, and a **personal space** that only you can read, not even administrators. You log in with your company account. Your assistants connect once. From then on, "check Atrium" is all you have to say.

That is the whole product: **write it down once, use it everywhere, in any assistant, by anyone who may.**

What it brings you

You stop repeating yourself. Put your background, preferences and recurring context into your personal space once. Any assistant in any chat can fetch it when you say so. "*Use my notes about the Phoenix project.*" No more pasting the same three paragraphs.

You pick up where you left off, in any assistant. Started a long analysis in one assistant and want a second opinion from another? "*Store the current state for ChatGPT.*" In the other assistant: "*Load my last handoff.*" Goal, results so far, open points and next steps are all there. Switching tools stops costing you the context.

You find answers instead of people. "*What does Atrium say about our travel policy?*" You get the relevant section with title, space, author and date, so you can check the source. Search works on meaning and on exact words, so you do not need to know the author's wording.

Your best prompts become reusable skills. The checklist you rebuild every Monday can become a skill in your personal space in minutes, available with `/name` from then on. If it is good, propose it to your team and it becomes the way everybody does it, after a curator has reviewed it.

Your notes get safer and better. Entries have versions and tags and can be archived instead of lost. Duplicates are caught before they pile up. Atrium rejects text that looks like passwords or keys before it is stored.

You stay in charge. An assistant files something in Atrium only when you ask. Moving, archiving and deleting need your confirmation. Your personal space is yours: nobody else can read it.

It costs you little. One connection per assistant, one sentence to save something, one sentence to find it again.

What it brings your team

A single source of truth, owned by people. The answer to "how do we do X?" lives in one place, written and kept current by the people who know, in the space of their team. When it changes, it changes for everybody.

Faster onboarding. A new colleague asks their assistant instead of five people. "*How do we handle a refund?*" The answer comes from your own policy, with the entry named, and the newcomer learns the vocabulary at the same time.

Fewer interruptions. The tenth question about the same thing gets answered by the entry, not by the colleague who answered the first nine. Experts spend their time on the new questions.

Decisions that survive. "*Document our decision about the API versioning in the Architecture space.*" Months later, "*Why did we choose that?*" has an answer, with a date and an author.

The same way of working for everyone. Skills turn "how we do it here" into something executable. A code review, a customer reply, an incident report: the same steps every time, in whichever assistant a colleague uses, with new versions rolled out to everybody at once.

Quality without bureaucracy. In spaces where it matters, new entries are drafts until a curator approves them. In spaces where it does not, authors publish directly. Skills always get a second pair of eyes before anyone can use them. You choose, space by space.

It works for the small and the large. A five-person team, a project group, a club, a family that shares recipes and paperwork: Atrium is for any group that wants one reliable memory. One Atrium serves one organisation, so "the group" can be as small or as large as you like.

One memory, many assistants

Most people end up using more than one assistant: one at the desk, another on the phone, a third for code, whatever a colleague prefers. Without a shared memory each of them knows only what you told it.

Atrium sits **underneath** all of them:

You use	What Atrium gives you
Claude	Search and file knowledge, load skills, use your handoffs
ChatGPT	The same knowledge, the same skills, the same handoffs
Claude Code and other MCP assistants	The same again, inside your development workflow
Several at once	One memory that none of them owns

This has four practical consequences:

1. **No lock-in.** Your knowledge is not trapped in one vendor's chat history. If a better assistant appears next year, you connect it and everything you saved is already there.
2. **Mixed teams work.** One colleague prefers Claude, another ChatGPT. They draw on the same entries and run the same skills.
3. **Personal continuity.** Your handoffs and notes follow *you*, not the chat window.
4. **Every device, nothing to sync.** The memory lives on your organisation's server, not on a device. Phone, laptop and the desk in the office all see the same state the moment you log in. There is nothing to install or update on the device, nothing to pull and nothing to merge, so it works the same for everybody, whatever their technical background.

What it brings the organisation

Knowledge that stays when people leave. Entries and skills belong to the organisation, not to a laptop. When someone leaves, the account is locked and, after a deadline, their personal space is deleted; team knowledge was never in their head only.

Consistency at scale. Skills mean that the quality of a customer reply or a review does not depend on who happens to do it. A fix to a procedure is one new version, reviewed once, available to all.

Control that matches the risk. Roles per space (reader, author, curator, admin), approval where wrong content would be costly, four-eyes review for skills, risk classes, the ability to block a skill at once, and an audit log of who did what. Personal spaces stay private even from administrators, which is a promise you can make to your staff.

Security you can explain. Login with your own identity provider (Google, Microsoft Entra or another OIDC provider) and a list of who may log in. Atrium runs on **your** server, under **your** control, with your backups and your database. Skills are text only: Atrium never executes anything. Content from entries is treated as data, never as instructions to an assistant.

Vendor independence. Atrium speaks the open Model Context Protocol. It works with several assistants today and does not tie your knowledge to any of them. You also choose the embedding service that powers search: your own server, an OpenAI-compatible service or Amazon Bedrock.

A faster return on every assistant licence. Assistants answer better when they know your context. Atrium is how they get it, for everybody, without each person doing the work of feeding it.

Visibility. Instance admins see what is being filed, reviewed, released and blocked, and can answer "who changed this?" and "who published that skill?" without detective work.

An illustration, not a measurement: if each of twenty colleagues loses ten minutes a week re-explaining context or hunting for a known answer, that is more than fourteen hours a month. The point is not the number, it is that this cost is invisible, constant and shared, and that a shared memory attacks exactly that.

A day with Atrium

08:45. You start a task. "*Check Atrium: what is our process for onboarding an enterprise customer?*" The assistant answers from the Sales space and names the entry and its date.

10:30. In a meeting you learn that the discount limit changed. "*File this in the Sales space: the discount limit is now 15 percent since 1 November, decided by the pricing board.*" The assistant saves it, tells you where, and warns if a similar entry exists. In a space with approval, a curator approves it.

13:00. A colleague asks how to answer a tricky ticket. You say: "*Run /support-reply on this.*" The same reviewed steps run for them, in their assistant, not yours.

15:30. You are deep in an analysis and want a second opinion from the other assistant. "*Store the current state for ChatGPT.*" There: "*Load my last handoff.*" You continue without a recap.

17:10. You notice that an old entry is wrong. You ask who looks after the space, tell the curator, and the correction is a new version, not a new rumour.

Friday. Your personal "weekly report" skill drafts the report in a minute. A colleague likes it, you propose it to the team, a curator reviews it, and from next week everybody has it.

None of these moments is large. Together they are the difference between assistants that help one person at a time and a team that gets smarter with every day.

Why you can trust it

- **You decide what is filed.** Nothing is saved on the assistant's initiative. Destructive actions ask you first.
- **Knowledge is data, skills are reviewed.** Text inside an entry can never steer an assistant. A skill is an instruction, so it is read by a curator, approved at a specific fingerprint (digest), released deliberately, and can be withdrawn at once.

- **Your private notes are private.** Nobody can read your personal space through Atrium, administrators included.
- **Rights are checked on every request.** Roles, locked accounts and revoked sessions take effect immediately. The check is part of the search itself: an assistant is never handed an entry that its user may not read, so nothing depends on the assistant behaving well.
- **Everything important leaves a trace.** Approvals, releases, blocks, moves and deletions are in the audit log.
- **No lock-in.** Your knowledge is plain Markdown text in a database you control and back up yourself, and you can take it out at any time: admins export a space or all team spaces, and everybody exports their own personal space, as a ZIP of Markdown files in the open Open Knowledge Format, with the metadata of every entry.

How Atrium compares

There are good ways to keep notes and knowledge. A popular one is a central Git repository with Markdown files in folders that everybody clones: simple, versioned and under your control. Others grew out of one person's notes (Obsidian) or one agent's memory (Hindsight) and can be shared within the limits of that model. Atrium was built from the start for a whole organisation and all the assistants its people use, and that is where it differs. The tools below are neighbours rather than rivals: your Git repository stays the home of your code and its documentation, your Obsidian vault stays your own thinking space, and the organisation gets one shared memory on top.

	Atrium	**Markdown files in a central Git repository**	**Obsidian**	**Hindsight**
Built for	A team or organisation and all its assistants	Code and documentation, here used as a shared folder for company knowledge	Personal notes, shareable as vaults	The memory of AI agents, per user or per team
Who can use it	Any assistant that speaks MCP: Claude, ChatGPT, Claude Code and others, on desktop and phone	Assistants that can read a local checkout, typically coding assistants	Assistants through third-party add-ons you set up yourself	Agents that you connect to it
One state for everybody	Yes: one server, nothing to clone, pull or merge	Everybody keeps a checkout and pulls; knowledge is as current as the last pull	Each vault lives on its own device; shared vaults through Obsidian Sync, each member with a subscription	A bank can be shared across tools, one per user or one per team

	Atrium	Markdown files in a central Git repository	Obsidian	Hindsight
Who may read what	Roles per space, tied to your user account so they follow you to every device; personal spaces private even from administrators; checked on every search	Per repository: all or nothing; finer rights mean more repositories	In a shared vault everybody has the same permissions; no fine-grained rights	Per memory bank; team knowledge separated by tags
Finding things	Search by meaning and exact words; the assistant gets the best matching sections with their source	File names and text search; the assistant reads whole files into its working memory	Text search in the vault	Its own retrieval over what it has extracted
Quality control	Drafts, curators, four-eyes review, audit log	Pull requests and reviews, if you set them up: natural for developers, a hurdle for everybody else	Your own discipline	Extracted automatically
Reviewed working instructions	Skills with approval, release and instant block	Files; everybody runs what they last pulled	No	No
What gets stored	Only what a person asks to be filed	Whatever is committed	Whatever you write	What the memory extracts from conversations
Setup	One Docker Compose file	A Git host, access rights, and a checkout on every device	None	A server plus a language model of its own
What it asks of hardware	One modest server, from about 2 GB RAM on two cores for the search service, more cores for faster imports; no language model, no GPU; nothing on the devices	A Git host, plus a checkout on every device	Nothing beyond the device; Sync is a paid service	A server, plus a language model for extracting facts: a cloud API (paid per use) or one you host yourself on suitable hardware
Across devices and phone	Same login, same state, same rights on every device; assistants on desktop and phone reach the same server	A checkout and credentials on every device; hardly practical on a phone	Apps for desktop and phone with sync; letting an assistant use the vault depends on add-ons, some of them desktop-only	Reachable over the network once the server runs; access is tied to keys or sign-in per bank rather than to one user account

What that means in practice:

- **A shared folder of Markdown, without the Git.** A central repository of Markdown files is the most common alternative to Atrium, and a fair one for a team of developers. For everybody else it means installing Git, cloning, pulling, resolving merge conflicts and living with rights per repository, while assistants outside the IDE cannot see it at all. Atrium keeps what is good about it, plain Markdown and versions, and removes the synchronising: you log in, and you and your assistants see the same state. Your content stays portable, because everything can be exported as Markdown files, for example into a repository.
- **One account, every device.** Your rights hang on your user account, not on a device, a key or a checkout. Log in on the laptop, the phone or a colleague's machine and you see the same spaces, the same roles and the same personal space. Lock the account and every device loses access at once.
- **Light on hardware, no language model to feed.** Atrium does not run a language model of its own. It needs only an embedding service for the search, which a modest server can host (from about 1.7 GB of RAM on two cores; more cores make bulk imports faster) or which you can take from a provider with no local memory needed at all. No GPU, and no token bill for filing something.
- **You decide what goes in.** Atrium does not learn behind your back. Nothing is filed unless a person asks for it, so passwords, private data and customer information do not end up in the shared memory by accident, and nothing is picked up from a chat that nobody meant to keep. Atrium also rejects text that looks like a password or key. This is full control by design.
- **One file, a few minutes.** Everything comes up from a single Docker Compose file. For an IT professional the setup takes minutes, not days. There is nothing to install on anybody's device.
- **The lowest possible threshold.** Filing something is a sentence in the chat you are already in, and it is available to everybody the moment it is saved. No new app, no new routine. That is how a shared memory grows: by being used, not by being maintained.
- **Bring what you already have.** With the shipped skill `knowledge-ingest`, an assistant that can read your Confluence can turn its pages into Atrium entries: "*Import the knowledge from the area Customer Service in Confluence into the space Customer Service in Atrium.*" You see what is proposed and confirm before anything is saved; from then on the space's admins and curators look after it. It costs some tokens once, and the knowledge stays.
- **Skills, not only notes.** Your best procedures become reviewed, versioned instructions that run the same way for everybody, in any assistant. A notes app or a repository keeps files; Atrium makes them usable.
- **Your knowledge stays yours.** It is plain Markdown in a database you control, and it can be exported at any time.

Use each tool for what it does best: Git for code and its documentation, Obsidian for your own thinking, a memory engine like Hindsight inside agents you build, and Atrium for what the whole organisation should know and what all its assistants should be able to find.

Product names are trademarks of their respective owners; Atrium is not affiliated with them. The comparison reflects publicly documented features as of November 2026.

Try it today: your first ten minutes

1. **Connect one assistant** with the address your administrator gives you, and log in with your company account.
2. **Ask:** "*Which Atrium spaces can I use, and what is my role?*"
3. **Search:** "*What does Atrium say about ?*" Check that the answer names its source.
4. **Save one thing** in your personal space: "*Save this in my Atrium space: <a checklist, a decision, a recurring context>.*"
5. **Use it in a new chat**, ideally in a second assistant: "*Use my Atrium notes about .*" This is the moment that makes it click.
6. **Try a handoff:** "*Store the current state for the other assistant,*" then load it there.
7. **Ask for the skills:** "*Which skills are available in Atrium?*" Run one with `/name`.

Then make it a habit with one rule: **whenever you explain something for the second time, put it in Atrium.** That single habit is where the value comes from.

For the details by role, continue with the guides: everyday use, authors, curators, space admins and instance admins.

The honest part: what Atrium does not do

Atrium is a memory, not magic. It helps you most when you are clear about what it is.

- **It is only as good as what is in it.** An empty Atrium helps nobody. That is why filing is one sentence in the chat you already use, and why you can bring existing knowledge in with the ingest skill instead of typing it again. The value grows with contributions, so the habit above matters more than any feature.
- **It does not replace judgement.** Assistants quote entries; entries can be outdated or wrong. Atrium shows the source and date so you can check, and curators exist to keep important spaces clean.
- **Tools use a little of an assistant's working memory.** Every connected tool brings a short description that the assistant has to keep in mind. Assistants that load tools only when they are needed use very little; others use more. A search returns the best matching sections, five by default and at most twenty, never whole libraries.
- **It does not run programs.** Skills are text. They guide an assistant; they do not execute code on their own.
- **Assistants differ.** How closely a given assistant follows a skill, or how it reacts to a `/name` command, depends on the assistant. The tools work in every MCP client; some clients are smoother than others.
- **Some conveniences are still missing.** There are no notifications yet (curators find drafts by asking), no browsing of old versions of an entry, and no attachments: entries are text. These are on the roadmap.

- **Privacy has a boundary.** Your personal space is private inside the application. Whoever administers the database or its backups could technically read everything, so your organisation should restrict that access and tell people the rules. Search uses an embedding service that your administrator chooses; with a cloud service, content is sent there to be indexed. What an assistant's vendor does with your chats is governed by that vendor's terms, not by Atrium.
- **It asks a little of you.** A sentence to file, a sentence to find, and the discipline to keep entries current. In return you get the compound interest of everybody doing the same.

What it takes to run

The **Community** edition is free for up to 8 active user accounts. It sends no telemetry and calls nobody. A small team can run it on one modest server. Everything comes together in one Docker Compose file, and for an IT professional the setup takes minutes:

- **Parts:** Atrium itself, a Postgres database with the pgvector extension, and an embedding service that makes the search understand meaning. They come together in one Docker Compose setup.
- **Memory and speed:** the embedding service is the largest part, and you choose how big it is. With the small-server settings it measured about 1.7 GB of RAM and 2.5 seconds per section on two cores, and about 2.1 GB and 1 second per section on four cores (details in embedding.md). More memory does not improve the results; more cores make bulk imports, such as a first import from a wiki, faster. The default arguments of the Compose file reserve about 12 GB, which Atrium's sections never fill, so set the small-server arguments in `.env`. Instead of running your own, you can connect an OpenAI-compatible service or Amazon Bedrock: no local memory is needed, it costs the provider's fees and it sends the text of entries to that provider.
- **Also needed:** a domain name with a TLS certificate, and a login through Google, Microsoft Entra or another OpenID Connect provider. Backups and updates are yours; the effort is small once it runs.
- **Platform:** the self-hosted embedding image runs on x86 servers only.

Details for administrators are in the README and the instance admin guide.

Rolling it out

For team leads and administrators, a pace that works:

When	What
Week 0	Install, connect the login, create two or three spaces, name two curators per space, set the instance context that tells assistants what Atrium is for
Week 1	A pilot group of five to ten people. Seed each space with ten genuinely useful entries and one or two skills. Start with what only a few long-serving colleagues know, and ask them to tell it to their assistant and file it
Week 2	Collect questions and fix the rough spots. Share this document and the everyday-use guide

When	What
Week 3 to 4	Roll out to the whole team. Ask everybody to file one thing a week. Curators review twice a week
Month 2 on	Review what is read and used. Retire what is stale, grow the skills that people actually run, and add spaces where there is demand

Three things make the difference: **seed it with real content**, **make the habit visible** (leaders file and use it first), and **keep curation light and quick** so that contributing never feels like paperwork.

Questions people ask

Is this just another wiki? A wiki waits for you to visit it. Atrium is where your assistants already are: you ask in the chat you are in, and the answer comes from your organisation's knowledge. It also carries skills and handoffs, which a wiki does not.

Why not just keep Markdown files in a central Git repository? For developers it works, and your code documentation belongs in the code repository anyway. As a company memory it asks every colleague to run Git, leaves browser and phone assistants out, hands whole files to the assistant and stops at rights per repository. Atrium keeps the Markdown and drops the synchronising, adds search, roles, review and skills, and exports back to Markdown at any time. Obsidian grew out of personal notes and Hindsight out of agent memory; both can be shared. Atrium adds roles per space, review, reviewed skills and one company login on top, for the step when a whole organisation and its different assistants need the same, access-controlled knowledge without anybody syncing anything. See How Atrium compares.

Will my personal notes be visible to my boss or to IT? No. Nobody can read a personal space through Atrium, administrators included. Team spaces are visible to their members, and administrators see all team spaces.

Does it work with the assistant I already use? Atrium works with assistants that support the Model Context Protocol, including Claude and ChatGPT. Connection steps differ slightly per product; your administrator provides the address and you log in with your company account.

Who can change what I file? In a team space, curators can correct entries; every change is a new version and the history is kept. In your personal space, only you.

What if an entry is wrong? Tell the curator of the space (the assistant can tell you who it is). A correction is a new version. Important spaces can require approval before anything becomes visible.

Do I need to learn commands? No. Describe what you want in your own words. The commands and tool names in the guides are for reference.

How much work is it? Connecting takes minutes. Filing or finding something is a sentence. The habit pays back the first time you do not have to explain something twice.

What happens to my knowledge if we change assistants? Nothing. It lives in Atrium, not in any assistant.

What if I leave the company? Your account is locked. Your personal space follows a documented procedure: you move team knowledge out beforehand, and an administrator decides about the rest.

Can I use it privately, with friends or family? One Atrium serves one organisation, and an organisation can be a family, a club or a circle of friends that runs its own instance. Each instance has its own login rules and its own spaces.

Where do I start? With step 1 of your first ten minutes, and with one promise to yourself: the next time you explain something for the second time, you put it in Atrium.